

Fashionable Data Pipeline

Analytics Engineering Assessment

Prepared by: Rhyando **Stack:** DuckDB, dbt, Metabase, Python

Press Space to begin →



Agenda

1. Requirements & Principles
2. How did you explore the data?
3. High-Level Architecture & Ingestion Flow
4. How did you clean the data?
5. Facts and Dimensions Reasoning
6. How did you set up dbt?
7. Preparing Data for BI & Answering
Marketeers
8. Metabase Example
9. How do you guarantee data quality?
10. Future Improvements
11. Thank You!

"Design a data warehouse using the Kimball star schema model, aimed at facilitating marketers' analytical tasks."

Requirements & Principles

To ensure the solution meets the business needs, the following core requirements guided the pipeline design:



Functional Requirements

- **Transformation:** Must use **dbt** and **SQL**.
- **Modeling:** Design a **Kimball Star Schema** data warehouse.
- **Integration:** Must be seamlessly connected to a **BI Tool** (Metabase).



Non-Functional Requirements

- **User Friendliness:** The data warehouse and BI layer must be highly intuitive, aimed at facilitating marketers' analytical tasks without friction.
- **Data Quality:** Need to ensure data quality for better trust from the marketer.

How did you explore the data?

Before building models, I needed to understand the shape, grain, and quality of the raw CSV.

Tools Used

- **Python (Pandas):** Initial load and inspection.
- **YData Profiling:** Automated statistical audit.
- **DuckDB:** Quick SQL ad-hoc queries.
- **Excel:** for quick raw data exploration

The profiling can be found here:
[ingestion/data_audit.html](https://www.ydata.ai/ingestion/data_audit.html)

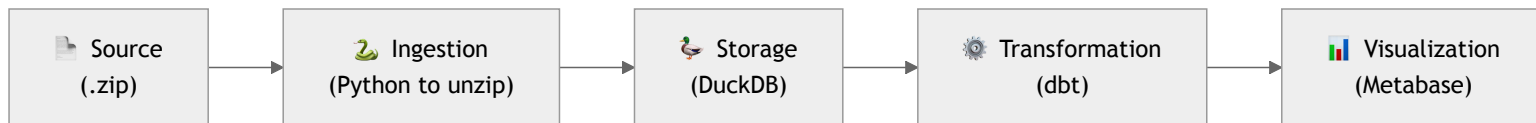
Key Findings

- **Grain:** One row per order line item.
- **Volume:** ~128,975 rows.
- **Data Issues:**
 - There is no unique row id and the column `index` is not a standard row id
 - `date` column in MM-DD-YY format is not standard and consistent.
 - Column `Unnamed: 22` has no actual meaning.
- **Missing Values:**
 - Missing `ship-city` fields.
 - 6.0% missing values in `Amount` .

💡 **Takeaway:** There are a lot of data quality implementation that hasn't been found. Only after more

High-Level Architecture & Ingestion Flow

The data pipeline is designed to be simple, reproducible, and fully local.



Workflow Breakdown:

1. **Source:** The assessment provided raw data in a compressed format.
2. **Ingestion:** A Python script automatically extracts the `.zip` file and loads the raw `.csv` into DuckDB.
3. **Transformation:** dbt takes over to clean, deduplicate, and model the data into a Kimball Star Schema.
4. **Visualization:** Metabase connects directly to the local DuckDB database to serve dashboards to the marketers.

How did you clean the data?

Data cleaning was handled systematically across two layers:

- **Python Ingestion:** mainly used as automatic pandas column schema and put data to duckdb.
- **dbt Staging (Silver):** Cleaning up mainly done in the silver layer.

List of changes:

- **Standardization:** Converted column names to `snake_case` during ingestion.
- **Data Types:**
 - Explicitly cast dates to `DATE` types in dbt from MM-DD-YY to normal YYYY-MM-DD.
 - Change format such as `Amount` to `FLOAT` and `ship-postal-code` to `INT`.
- **Handling Nulls:** Monitored missing columns such as `Amount` values (6%). Imputed or filtered where necessary.
- **Deduplication:** Applied window functions in DuckDB to eliminate duplicated order lines.
- **Remove:** Removing the first date of the data due to unusually low value.

Facts and Dimensions Reasoning

To support the marketing team, I designed a **Kimball Star Schema** to make querying intuitive and performant.

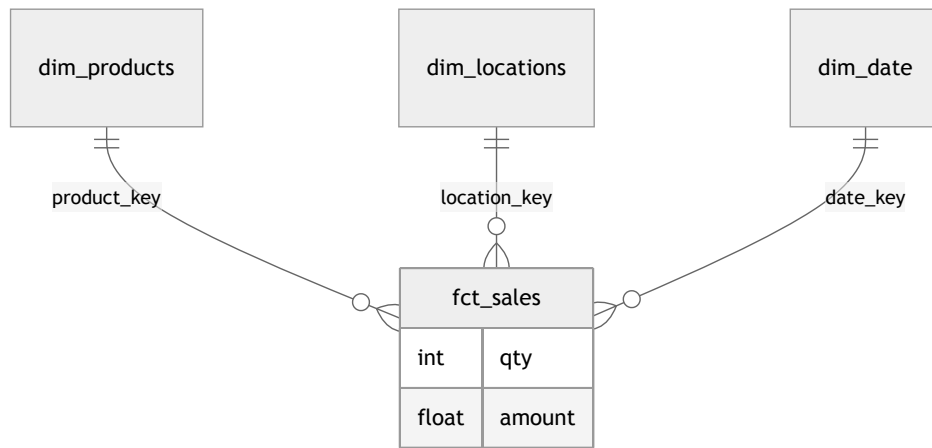
Fact Table

`fct_sales`

- **Grain:** One row per order line item.
- **Measures:** Quantity, Amount.
- **Keys:** `order_id` , `product_key` , `location_key` , `date_key` .

Dimension Tables

- `dim_products` : Product attributes (`sku` , `category` , `style` , `size` , etc).
- `dim_locations` : Shipping destinations (`ship_city` , `ship_state` , etc).
- `dim_date` : Calendar attributes (`month` , `quarter` , `season` , etc).



How did you set up dbt?

The dbt project is organized following dbt Labs' best practices.

- **Adapter:** `dbt-duckdb` for fast, local, analytical querying.
- **Sources (Bronze Layer):** Defined `source.yml` pointing to raw data already in DuckDB. It also include `stg_` which is minor adjustment.
- **Silver Layer:** `dim_`, and `fct_` models to clean, deduplicate, and build the core star schema.
- **Golden Layer (Marts):** `mart_` models containing business-level aggregations directly for BI.

```
dbt/models/  
├── bronze/  
│   ├── _silver_models.yml  
│   └── stg_sales.sql  
├── silver/  
│   ├── dim_locations.sql  
│   ├── dim_date.sql  
│   ├── dim_products.sql  
│   └── fct_sales.sql  
└── gold/  
    ├── mart_sales.sql  
    └── obt_sales.sql
```


Preparing Data for BI & Answering Marketeers

The dimensional model directly answers the core business questions.

Q: Popular in Mumbai?

```
Filter dim_locations.ship_city = 'MUMBAI' .  
Group by dim_products.style and category , summing  
fct_sales.qty .
```

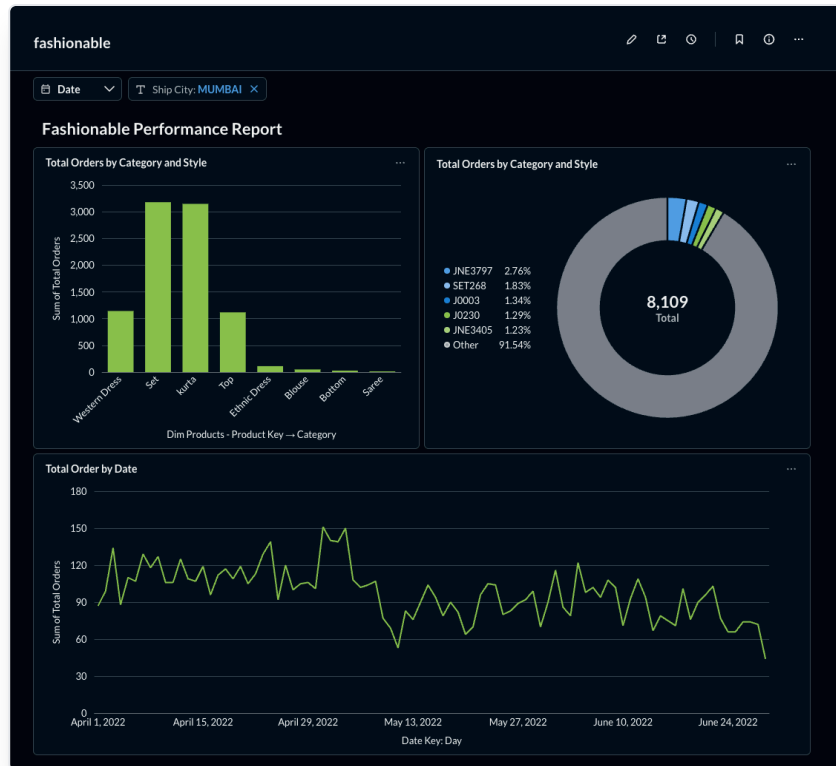
Q: Seasonal Sales Trend?

```
Join dim_date and group by season .  
Sum fct_sales.amount to visualize gross revenue trends.
```

BI Integration (Metabase):

- Connected Metabase directly to the DuckDB file.
- Created fact and dimension in silver layer for marketer who want to tap into the raw table.
- Created `mart_sales` in gold layer containing aggregated data from raw tables.
- Created a denormalized "One Big Table" (`obt_sales`) view to make self-serve BI exploration easier for marketers without requiring complex joins.

Metabase Example



How do you guarantee data quality?

Implemented robust testing in dbt to ensure data reliability.

- **Primary Key Tests:** `unique` and `not_null` on `order_id` + `sku` in staging, and surrogate keys in marts.
- **Referential Integrity:** `relationships` tests to ensure `fct_sales` keys exist in dimension tables.
- **Accepted Values:** Ensured order status and categories fall within expected static lists.
- **Custom Warnings:** Set severity to `warn` for the missing 6% `Amount` data, acknowledging the historical gap without failing the pipeline.

This data quality is implemented inside dbt test but not enforced right now.

Future Improvements

Given more than 4-6 hours, here is how I would evolve this architecture for scale:

1. 🚀 **Incremental Models:** Transition `fact_sales` from a full `table` rebuild to an `incremental` materialization to optimize processing time and compute costs as data volume grows.
2. 🤖 **CI/CD Automation:** Implement GitHub Actions to enforce data quality by running `dbt test` and `dbt build` automatically on every Pull Request.
3. 📖 **Data Cataloging & Lineage:** Generate and host `dbt docs` to provide marketers and analysts with a self-serve, searchable data dictionary and visual lineage graph.
4. ☁️ **Cloud DWH Migration:** Transition from a local DuckDB instance to a cloud-native platform like Snowflake or BigQuery to support multi-user concurrency and larger datasets.
5. ✂️ **Advanced Data Cleansing:** Implement more rigorous data quality checks and anomaly detection, such as fuzzy matching for missing or misspelled city names.
6. 📝 **Comprehensive Documentation:** Expand model-level and column-level descriptions in `schema.yml` to ensure business logic is fully transparent and maintainable.
7. **Additional Data Source:** Marketer's ask about seasonal could not be solved because there is no external data source.
8. **Proper BI Tool:** Move away from metabase to the more user friendly (paid) one such as PowerBI or Tableau

Thank You!

Questions?

AI (Gemini) is being used mainly for brainstorming, doing repetitive stuff, and this documentation.

Powered by  Slidev